Programación estructurada

La programación estructurada en PHP es un enfoque de programación que se basa en la creación de programas utilizando una estructura clara y organizada. Este enfoque se centra en la utilización de técnicas que permiten dividir un programa en pequeñas partes o módulos para facilitar su comprensión, modificación y mantenimiento.

La programación estructurada en PHP se basa en tres elementos fundamentales: secuencias, decisiones y bucles. Estos elementos permiten crear programas que realizan operaciones de forma ordenada y controlada.

La secuencia se refiere a la ejecución de instrucciones de forma secuencial, es decir, una tras otra. La decisión se utiliza para realizar diferentes acciones en función de una condición. Por último, el bucle se utiliza para repetir un conjunto de instrucciones varias veces.

En PHP, la programación estructurada se realiza mediante el uso de funciones y procedimientos. Estas son unidades de código que realizan una tarea específica y que pueden ser llamadas desde cualquier parte del programa. Al dividir un programa en funciones y procedimientos, se logra una mayor modularidad y se facilita su comprensión y mantenimiento.

Además, la programación estructurada en PHP también se beneficia de la utilización de variables y constantes bien definidas y documentadas. Esto permite una mayor claridad en el código y una menor propensión a errores.

La programación estructurada en PHP es un enfoque de programación que busca la creación de programas claros, organizados y fáciles de entender. Este enfoque se basa en el uso de secuencias, decisiones y bucles, así como en la utilización de funciones y procedimientos, variables y constantes bien definidas y documentadas.

Ejemplo de aplicación:

Supongamos que queremos crear un programa que permita calcular el área y perímetro de un rectángulo. En lugar de escribir todo el código en un solo bloque, podemos dividir el programa en tres partes principales: la entrada de datos, el cálculo y la salida de los resultados.

1.Entrada de datos:
Para solicitar los datos del usuario, podemos crear un formulario en HTML que permita ingresar la base y la altura del rectángulo. Luego, podemos utilizar la superglobal $_POST en PHP para obtener los valores ingresados por el usuario.

```php
<?php
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $base = $_POST['base'];
    $altura = $_POST['altura'];
}
?>
```

2.Cálculo:
Una vez que tenemos los valores de la base y la altura, podemos calcular el área y el perímetro del rectángulo. Para ello, podemos crear dos funciones en PHP que realicen estos cálculos.

```php
<?php
function calcularArea($base, $altura) {
    $area = $base * $altura;
    return $area;
}

function calcularPerimetro($base, $altura) {
    $perimetro = 2 * ($base + $altura);
    return $perimetro;
}
?>
```

3.Salida de resultados:
Finalmente, podemos mostrar los resultados en pantalla utilizando una estructura HTML y PHP que muestre los valores calculados.

```
<?php
if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $base = $_POST['base'];
    $altura = $_POST['altura'];

    $area = calcularArea($base, $altura);
    $perimetro = calcularPerimetro($base, $altura);

    echo "<p>El área del rectángulo es: $area</p>";
    echo "<p>El perímetro del rectángulo es: $perimetro</p>";
}
?>
```

En este ejemplo, hemos utilizado las técnicas de programación estructurada para dividir el programa en partes más pequeñas y manejables. Hemos utilizado funciones para realizar los cálculos y hemos separado la entrada y salida de datos del cálculo en sí mismo. Esto hace que el código sea más fácil de entender, mantener y modificar en el futuro.

Tipos de variables

En PHP existen varios tipos de variables que se utilizan para almacenar diferentes tipos de datos. A continuación, se muestran los tipos de variables más comunes en PHP con algunos ejemplos de cómo se pueden utilizar:

1.Variables numéricas:
Las variables numéricas se utilizan para almacenar números enteros o decimales. En PHP, no es necesario especificar el tipo de datos al definir una variable numérica, ya que PHP detecta automáticamente el tipo de datos.

Ejemplo:

```
$num_entero = 10;
$num_decimal = 3.14;
```

2.Variables de texto:
Las variables de texto se utilizan para almacenar cadenas de caracteres. En PHP, las cadenas de caracteres se pueden definir utilizando comillas simples o dobles.

Ejemplo:

```
$texto_simple = 'Hola, mundo!';
$texto_doble = "¡Bienvenido al curso de desarrollo web!";
```

3.Variables booleanas:
Las variables booleanas se utilizan para almacenar valores de verdadero o falso. En PHP, el valor verdadero se representa con la palabra "true" y el valor falso con la palabra "false".

Ejemplo:

```
$verdadero = true;
$falso = false;
```

4.Variables de arreglo:
Las variables de arreglo se utilizan para almacenar una colección de valores relacionados. En PHP, los arreglos se pueden definir utilizando la función "array()".

Ejemplo:

```
$colores = array("rojo", "verde", "azul");
$numeros = array(1, 2, 3, 4, 5);
```

Arreglos con índices asociativos: se utilizan para asociar un valor con una clave en lugar de un índice numérico. Ejemplos de cómo crear arreglos con índices asociativos en PHP:

Ejemplo 1:

```
// Declaración del arreglo asociativo
$personas = array(
    "nombre" => "Juan",
    "apellido" => "Pérez",
    "edad" => 30,
    "profesion" => "Ingeniero"
);

// Acceso a los elementos del arreglo
echo "Nombre: " . $personas["nombre"] . "<br>";
echo "Apellido: " . $personas["apellido"] . "<br>";
echo "Edad: " . $personas["edad"] . "<br>";
echo "Profesión: " . $personas["profesion"] . "<br>";
```

Ejemplo 2:

```
// Declaración del arreglo asociativo
$productos = array(
    "producto1" => array(
        "nombre" => "Laptop",
        "precio" => 800,
        "disponible" => true
    ),
    "producto2" => array(
        "nombre" => "Smartphone",
        "precio" => 500,
        "disponible" => false
    ),
    "producto3" => array(
        "nombre" => "Tablet",
        "precio" => 400,
        "disponible" => true
    )
);

// Acceso a los elementos del arreglo
echo "Nombre del producto 1: " . $productos["producto1"]["nombre"] . "<br>";
echo "Precio del producto 1: " . $productos["producto1"]["precio"] . "<br>";
echo "Disponible: " . ($productos["producto1"]["disponible"] ? "Sí" : "No") . "<br>";
```

En este ejemplo, hemos creado un arreglo asociativo multidimensional llamado $productos que contiene los datos de varios productos. Los elementos del arreglo se pueden acceder utilizando las claves asociativas de cada producto.

Los arreglos con índices asociativos son útiles en PHP porque permiten asociar un valor con una clave en lugar de un índice numérico. Esto facilita el acceso y la manipulación de los datos en el arreglo.

5.Variables nulas:

Las variables nulas se utilizan para indicar que una variable no tiene ningún valor. En PHP, se puede definir una variable como nula utilizando la palabra clave "null".

Ejemplo:

```
$nulo = null;
```

Estructura condicional if

se utiliza para ejecutar un bloque de código si se cumple una condición. La sintaxis básica de la estructura if es la siguiente:

```
if (condición) {
  // Bloque de código a ejecutar si se cumple la condición
}
```

Ejemplos de cómo se puede utilizar la estructura condicional if en PHP

Ejemplo 1:

```
$edad = 20;

if ($edad >= 18) {
  echo "Eres mayor de edad";
}
```

En este ejemplo, hemos utilizado la estructura if para verificar si la variable $edad es mayor o igual a 18. Si la condición se cumple, se ejecutará el bloque de código que imprime "Eres mayor de edad" en la pantalla.

Ejemplo 2:

```
$numero = 5;

if ($numero % 2 == 0) {
  echo "El número es par";
} else {
  echo "El número es impar";
}
```

En este ejemplo, hemos utilizado la estructura if para verificar si la variable $numero es divisible por 2. Si el resultado es cero (es decir, si el número es par), se ejecutará el primer bloque de código. De lo contrario, se ejecutará el segundo bloque de código.

Ejemplo 3:

```
$usuario = "admin";
$password = "1234";

if ($usuario == "admin" && $password == "1234") {
  echo "Bienvenido, has iniciado sesión correctamente";
} else {
  echo "El usuario o la contraseña son incorrectos";
}
```

En este ejemplo, hemos utilizado la estructura if para verificar si el usuario y la contraseña son correctos. Si se cumple la condición, se ejecutará el primer bloque de código que imprime "Bienvenido, has iniciado sesión correctamente". De lo contrario, se ejecutará el segundo bloque de código que imprime "El usuario o la contraseña son incorrectos".

Ejemplos utilizando variables booleanas
Las variables booleanas son variables que pueden tomar solo dos valores posibles: true o false. Estas variables son muy útiles en las estructuras condicionales, ya que nos permiten verificar si se cumple o no una determinada condición.

Ejemplo de cómo se puede utilizar una variable booleana en una estructura condicional if en PHP:

```
$usuario_autenticado = true;

if ($usuario_autenticado) {
  echo "Bienvenido, has iniciado sesión correctamente";
} else {
  echo "Por favor, inicia sesión para acceder a tu cuenta";
}
```

En este ejemplo, hemos creado una variable booleana llamada $usuario_autenticado que toma el valor true. Luego, utilizamos la estructura if para verificar si la variable es verdadera. Si se cumple la condición, se ejecutará el bloque de código que imprime "Bienvenido, has iniciado sesión correctamente". De lo contrario, se ejecutará el segundo bloque de código que imprime "Por favor, inicia sesión para acceder a tu cuenta".

Estructura repetitivas for, while y do-while

Estas estructuras se utilizan para ejecutar un bloque de código varias veces, mientras se cumpla una determinada condición. A continuación, algunos ejemplos de cómo funcionan estas estructuras:

Estructura repetitiva for:

La estructura for se utiliza cuando se sabe de antemano la cantidad de veces que se debe ejecutar el bloque de código. La sintaxis básica de la estructura for es la siguiente:

```
for (inicialización; condición; incremento/decremento) {
  // Bloque de código a ejecutar en cada iteración
}
```

En la inicialización, se establece el valor inicial de la variable de control. En la condición, se verifica si se cumple la condición para continuar ejecutando el bloque de código. En el incremento o decremento, se modifica el valor de la variable de control en cada iteración.

Ejemplo:

```
for ($i = 0; $i <= 10; $i++) {
  echo $i . "<br>";
}
```

En este ejemplo, se utiliza la estructura for para imprimir los números del 0 al 10 en pantalla. La variable $i se inicializa en 0, la condición establece que se debe seguir ejecutando el bloque de código mientras $i sea menor o igual a 10, y en cada iteración se incrementa el valor de $i en 1.

Estructura repetitiva while:

La estructura while se utiliza cuando no se sabe de antemano la cantidad de veces que se debe ejecutar el bloque de código. La sintaxis básica de la estructura while es la siguiente:

```
while (condición) {
  // Bloque de código a ejecutar mientras se cumpla la condición
}
```

En este caso, el bloque de código se ejecutará mientras se cumpla la condición. Es importante asegurarse de que en algún momento la condición deje de ser verdadera, de lo contrario se producirá un bucle infinito.

Ejemplo:

```
$i = 0;

while ($i < 5) {
  echo $i . "<br>";
  $i++;
}
```

En este ejemplo, se utiliza la estructura while para imprimir los números del 0 al 4 en pantalla. La variable $i se inicializa en 0, y se incrementa en cada iteración. El bloque de código se ejecutará mientras $i sea menor que 5.

Estructura repetitiva do-while:

La estructura do-while es similar a la estructura while, con la diferencia de que el bloque de código se ejecuta al menos una vez, antes de comprobar la condición. La sintaxis básica de la estructura do-while es la siguiente:

```
do {
  // Bloque de código a ejecutar al menos una vez
} while (condición);
```

Ejemplo:

```
$i = 0;

do {
  echo $i . "<br>";
  $i++;
} while ($i < 5);
```

En este ejemplo, se inicializa la variable $i con el valor 0. Luego, se utiliza la estructura do-while para imprimir los valores de $i en pantalla. El bloque de código dentro del do se ejecutará al menos una vez, independientemente de si la condición se cumple o no. En cada iteración, se incrementa el valor de $i en 1. La condición establece que el ciclo debe continuar mientras $i sea menor que 5.

Introducción a los archivos

Los archivos son una parte fundamental en el desarrollo de aplicaciones web. Son utilizados para almacenar y manipular información de manera persistente.

Un archivo es un conjunto de datos guardados en un dispositivo de almacenamiento, como el disco duro de una computadora.

En programación, los archivos nos permiten guardar datos estructurados de forma que puedan ser utilizados posteriormente, incluso cuando el programa se haya cerrado o reiniciado.

Los archivos pueden contener diferentes tipos de datos, como texto, imágenes, videos o cualquier otro tipo de información que se pueda almacenar digitalmente.

Los archivos son esenciales para tareas como almacenar configuraciones, guardar registros de actividad, interactuar con bases de datos y manipular contenido multimedia.

Los archivos se puede utilizar en situaciones como guardar el historial de un juego, almacenar información de usuarios registrados en una aplicación o guardar registros de ventas en un sistema de inventario; Además, algunos escenarios comunes en los que se utilizan archivos en el desarrollo web son; leer y escribir datos en archivos de texto, manipular archivos CSV para el procesamiento de datos o almacenar imágenes y archivos multimedia en servidores.

Creación y apertura de archivos:

En PHP, puedes crear y abrir archivos utilizando la función **fopen()**.
La función **fopen()** acepta dos parámetros principales: la ruta del archivo y el modo de apertura.

La ruta del archivo puede ser una ruta relativa (por ejemplo, "archivos/miarchivo.txt") o una ruta absoluta (por ejemplo, "/var/www/html/archivos/miarchivo.txt").

El modo de apertura define cómo se manejará el archivo. Algunos modos comunes son:

- "r": Abre el archivo en modo de lectura. El puntero del archivo se coloca al principio.
- "w": Abre el archivo en modo de escritura. Si el archivo existe, se truncará (es decir, se eliminará el contenido existente). Si no existe, se creará uno nuevo.
- "a": Abre el archivo en modo de anexado (escritura al final). Si el archivo existe, el puntero se coloca al final del archivo. Si no existe, se creará uno nuevo.
- Por ejemplo, para crear un nuevo archivo llamado "miarchivo.txt" en modo de escritura, puedes usar fopen("archivos/miarchivo.txt", "w").

Es importante verificar si la función **fopen()** devuelve un valor válido, ya que puede fallar si no se tienen los permisos adecuados o si la ruta del archivo es incorrecta.

Ejemplo práctico

```
<?php
$archivo = fopen("archivos/miarchivo.txt", "w");

if ($archivo) {
    fwrite($archivo, "¡Hola, mundo!");
    fclose($archivo);
    echo "Se ha creado y escrito en el archivo exitosamente.";
} else {
    echo "No se pudo abrir el archivo.";
}
?>
```

En este ejemplo, estamos creando un nuevo archivo llamado "miarchivo.txt" en el directorio "archivos" y abriéndolo en modo de escritura ("w"). Si el archivo se abre correctamente, utilizamos la función **fwrite()** para escribir el texto "¡Hola, mundo!" en el archivo. Luego, cerramos el archivo con **fclose()**. Finalmente, mostramos un mensaje indicando si el archivo se creó y escribió correctamente.
Recuerda asegurarte de tener los permisos adecuados en el directorio donde deseas crear el archivo y que la ruta del archivo sea correcta en relación con el archivo PHP que estás ejecutando.

Lectura y escritura de archivos:
En PHP, existen varias funciones **para leer** y escribir en archivos. Para leer el contenido de un archivo, puedes utilizar la función **fgets()** o **file_get_contents()**.
fgets() lee una línea completa del archivo y mueve el puntero al siguiente.
file_get_contents() lee todo el contenido del archivo y lo devuelve como una cadena de texto.
Por ejemplo, para leer el contenido de un archivo línea por línea con fgets():

```
<?php
$archivo = fopen("archivos/miarchivo.txt", "r");

if ($archivo) {
    while (($linea = fgets($archivo)) !== false) {
        echo $linea . "<br>";
    }
    fclose($archivo);
} else {
    echo "No se pudo abrir el archivo.";
}
?>
```

En este ejemplo, abrimos el archivo "miarchivo.txt" en modo de lectura ("r") y utilizamos un bucle while para leer y mostrar cada línea del archivo utilizando la función **fgets().** Luego, cerramos el archivo con **fclose()**.

Para **escribir** en un archivo, puedes utilizar las funciones **fwrite()** o **file_put_contents()**.
fwrite() te permite escribir una cadena de texto o datos en el archivo.
file_put_contents() reemplaza todo el contenido del archivo con una cadena de texto.

```
<?php
$archivo = fopen("archivos/miarchivo.txt", "w");

if ($archivo) {
    fwrite($archivo, "¡Hola, mundo!");
    fclose($archivo);
    echo "Se ha escrito en el archivo exitosamente.";
} else {
    echo "No se pudo abrir el archivo.";
}
?>
```

En este ejemplo, abrimos el archivo "miarchivo.txt" en modo de escritura ("w") y utilizamos **fwrite()** para escribir el texto "¡Hola, mundo!" en el archivo. Luego, cerramos el archivo con **fclose()**.

Recuerda verificar si la apertura del archivo fue exitosa y manejar los posibles errores que puedan surgir al leer o escribir en el archivo.

Para verificar posibles errores al leer o escribir en un archivo en PHP, puedes hacer uso de la función ferror() y comprobar si devuelve un valor distinto de cero. Aquí tienes un ejemplo:

```
$archivo = fopen("archivos/miarchivo.txt", "r");

if ($archivo) {
    // Operaciones de lectura o escritura
    // ...
    fclose($archivo);
} else {
    echo "No se pudo abrir el archivo.";
}

if (ferror($archivo)) {
    echo "Ocurrió un error al leer o escribir en el archivo.";
}
```

En el ejemplo, después de realizar las operaciones de lectura o escritura en el archivo, se utiliza **ferror($archivo)** para verificar si se ha producido algún error. Si **ferror()** devuelve un valor distinto de cero, significa que se ha producido un error al leer o escribir en el archivo. En ese caso, puedes mostrar un mensaje indicando que ocurrió un error.

Es importante tener en cuenta que la función **ferror()** solo debe llamarse después de realizar las operaciones de lectura o escritura en el archivo y antes de cerrarlo con fclose(), ya que una vez que se cierra el archivo, no se puede acceder a su estado de error.

También se puede utilizar otras funciones y técnicas para manejar los errores, como excepciones o estructuras de control como try-catch. Esto dependerá de tus preferencias y de la complejidad del código que estés desarrollando.

Ejemplo utilizando la estructura de control try-catch para manejar errores con excepciones:

```
try {
    $archivo = fopen("archivos/miarchivo.txt", "r");

    if (!$archivo) {
        throw new Exception("No se pudo abrir el archivo.");
    }

    // Operaciones de lectura o escritura
    // ...

    fclose($archivo);
} catch (Exception $e) {
    echo "Error: " . $e->getMessage();
}
```

En este ejemplo, utilizamos la estructura **try-catch** para encapsular el bloque de código que puede generar una excepción. Dentro del bloque try, abrimos el archivo "miarchivo.txt" en modo de lectura y realizamos las operaciones necesarias. Si ocurre algún error durante la apertura o las operaciones, lanzamos una excepción utilizando **throw new Exception("mensaje de error")**.

Fuera del bloque **try**, capturamos la excepción con la estructura **catch**. En este caso, capturamos la excepción de tipo **Exception** y accedemos al mensaje de error utilizando **$e->getMessage()**. Luego, mostramos el mensaje de error utilizando **echo**.

Utilizar excepciones te brinda un mayor control y flexibilidad para manejar los errores de manera específica y personalizada. Puedes lanzar diferentes tipos de excepciones según el tipo de error y capturarlas en diferentes bloques **catch** para manejar cada situación de manera adecuada.

Cierre y manipulación de archivos

Es importante cerrar los archivos después de trabajar con ellos para liberar los recursos del sistema y garantizar la integridad de los datos.
Para cerrar un archivo en PHP, se utiliza la función **fclose()**, que toma como parámetro el identificador del archivo abierto.
Por ejemplo, para cerrar un archivo:

```
$archivo = fopen("archivos/miarchivo.txt", "r");

// Realizar operaciones de lectura o escritura

fclose($archivo);
```

Además del cierre de archivos, existen otras funciones útiles para manipular archivos en PHP:

- **file_exists():** Verifica si un archivo existe en la ruta especificada.
- **rename():** Cambia el nombre de un archivo.
- **unlink():** Elimina un archivo.

```
if (file_exists("archivos/miarchivo.txt")) {
    rename("archivos/miarchivo.txt", "archivos/nuevoarchivo.txt");
    unlink("archivos/antiguoarchivo.txt");
}
```

En este ejemplo, primero verificamos si el archivo "miarchivo.txt" existe utilizando **file_exists().** Luego, utilizamos **rename()** para cambiar el nombre del archivo a "nuevoarchivo.txt". Por último, utilizamos **unlink()** para eliminar el archivo "antiguoarchivo.txt".

Recuerda siempre realizar verificaciones de seguridad antes de realizar operaciones como cambiar nombres o eliminar archivos. Puedes utilizar estructuras condicionales, como **if**, para verificar si el archivo existe o si la operación se llevó a cabo correctamente antes de continuar.

Estas funciones de manipulación de archivos te permitirán realizar tareas adicionales, como renombrar archivos, moverlos a diferentes ubicaciones o eliminar archivos innecesarios.

Ejemplo de cómo mover un archivo utilizando la función rename() en PHP

```
$archivoOrigen = "archivos/miarchivo.txt";
$archivoDestino = "archivos/carpeta/miarchivo.txt";

if (file_exists($archivoOrigen)) {
    if (rename($archivoOrigen, $archivoDestino)) {
        echo "El archivo se ha movido correctamente.";
    } else {
        echo "No se pudo mover el archivo.";
    }
} else {
    echo "El archivo de origen no existe.";
}
```

En este ejemplo, tenemos un archivo llamado "miarchivo.txt" que se encuentra en la ubicación de **$archivoOrigen**. Utilizamos **file_exists()** para verificar si el archivo de origen existe.

Si el archivo de origen existe, utilizamos **rename()** para moverlo a la ubicación especificada en **$archivoDestino**. Si **rename()** devuelve true, significa que el archivo se movió correctamente. De lo contrario, mostramos un mensaje indicando que no se pudo mover el archivo.

Manipulación de directorios

Además de trabajar con archivos individuales, PHP también proporciona funciones para manipular directorios (carpetas) en el sistema de archivos.

Algunas funciones útiles para la manipulación de directorios son:

- **mkdir():** Crea un nuevo directorio.
- **rmdir():** Elimina un directorio vacío.
- **opendir():** Abre un directorio y obtiene un identificador para su lectura.
- **readdir():** Lee el contenido de un directorio.
- **closedir():** Cierra un directorio abierto.

```
$directorio = "archivos/nuevodirectorio";

// Crear un directorio
if (!file_exists($directorio)) {
    if (mkdir($directorio, 0755)) {
        echo "El directorio se ha creado correctamente.";
    } else {
        echo "No se pudo crear el directorio.";
    }
} else {
    echo "El directorio ya existe.";
}
// Leer el contenido de un directorio
$contenido = opendir($directorio);
while (($archivo = readdir($contenido)) !== false) {
    echo $archivo . "<br>";
}
closedir($contenido);

// Eliminar un directorio vacío
if (rmdir($directorio)) {
    echo "El directorio se ha eliminado correctamente.";
} else {
    echo "No se pudo eliminar el directorio.";
}
```

En este ejemplo, primero verificamos si el directorio **$directorio** existe utilizando **file_exists()**. Si no existe, utilizamos **mkdir()** para crear un nuevo directorio con los **permisos 0755**. Si **mkdir()** devuelve true, significa que el directorio se creó correctamente.

Luego, utilizamos **opendir()** para abrir el directorio y obtener un identificador para leer su contenido. Mediante un bucle **while** y **readdir()**, leemos y mostramos el contenido del directorio. Finalmente, cerramos el directorio con **closedir()**.

Después de leer el contenido del directorio, utilizamos **rmdir()** para eliminar el directorio vacío. Si **rmdir()** devuelve true, significa que el directorio se eliminó correctamente.

Recuerda que al eliminar un directorio con **rmdir()**, este debe estar vacío. Si deseas eliminar un directorio que contiene archivos, primero debes eliminar los archivos individualmente o utilizar funciones adicionales para realizar una eliminación recursiva del directorio y su contenido.

Es importante tener precaución al manipular directorios y asegurarse de tener los permisos adecuados para crear, leer y eliminar directorios en el sistema de archivos.

subir un archivo mediante un formulario HTML y PHP

HTML del formulario:

- Crea un formulario HTML con el atributo **enctype="multipart/form-data"**.
- Agrega un campo **<input type="file">** para permitir al usuario seleccionar el archivo a subir.
- Puedes incluir otros campos y elementos en el formulario según tus necesidades.

```
<form action="subir_archivo.php" method="POST" enctype="multipart/form-data">
    <input type="file" name="archivo">
    <input type="submit" value="Subir archivo">
</form>
```

PHP para procesar la subida del archivo:

- Crea un archivo PHP (por ejemplo, "subir_archivo.php") para procesar el formulario.
- Utiliza la superglobal **$_FILES** para acceder a la información del archivo subido.
- Puedes realizar verificaciones y validaciones adicionales según tus necesidades.

```php
<?php
if (isset($_FILES['archivo'])) {
    $nombreArchivo = $_FILES['archivo']['name'];
    $tipoArchivo = $_FILES['archivo']['type'];
    $tamanoArchivo = $_FILES['archivo']['size'];
    $rutaArchivo = $_FILES['archivo']['tmp_name'];
    $errorArchivo = $_FILES['archivo']['error'];

    // Realizar verificaciones y validaciones adicionales si es necesario

    if ($errorArchivo === UPLOAD_ERR_OK) {
        $directorioDestino = "archivos/";
        $rutaDestino = $directorioDestino . $nombreArchivo;

        if (move_uploaded_file($rutaArchivo, $rutaDestino)) {
            echo "El archivo se ha subido correctamente.";
        } else {
            echo "No se pudo subir el archivo.";
        }
    } else {
        echo "Ocurrió un error al subir el archivo.";
    }
}
?>
```

En este ejemplo, el formulario HTML apunta al archivo "subir_archivo.php" como la URL de acción. En PHP, se verifica si existe **$_FILES['archivo']** para asegurarse de que se haya enviado un archivo.

Se accede a la información del archivo mediante los índices del array **$_FILES**. Puedes obtener el nombre del archivo, el tipo, el tamaño, la ruta temporal y el código de error, si lo hubiera.

A continuación, puedes realizar verificaciones y validaciones adicionales según tus necesidades. Luego, se verifica si no hubo errores al subir el archivo **($errorArchivo === UPLOAD_ERR_OK)** y se especifica el directorio de destino donde se almacenará el archivo.

La función **move_uploaded_file()** se utiliza para mover el archivo temporal a la ubicación de destino especificada. Si se mueve correctamente, se muestra un mensaje de éxito. De lo contrario, se muestra un mensaje de error.

Recuerda que debes asegurarte de tener los permisos adecuados en el directorio de destino para poder subir y guardar el archivo.

Ejercitación

Ejercicio 1: Sumar dos números
Escribe un script que pida al usuario dos números, los sume y muestre el resultado en pantalla.

Ejercicio 2: Calcular el promedio
Escribe un script que pida al usuario tres notas y calcule el promedio de las mismas. Luego, muestra el resultado en pantalla.

Ejercicio 3: Mostrar los números pares
Escribe un script que muestre en pantalla todos los números pares del 1 al 20.

Ejercicio 4: Mostrar la tabla de multiplicar
Escribe un script que pida al usuario un número y muestre en pantalla su tabla de multiplicar, del 1 al 10.

Ejercicio 5: Contar las vocales
Escribe un script que pida al usuario una cadena de texto y cuente cuántas vocales tiene.

Ejercicio 6: Mostrar números primos
Escribe un script que muestre en pantalla los primeros 10 números primos.

Ejercicio 7: Ordenar un arreglo
Escribe un script que tome un arreglo de números y los ordene de menor a mayor.

Ejercicio 8: Encontrar el número más grande
Escribe un script que tome un arreglo de números y encuentre el número más grande.

Ejercicio 9: Generar un arreglo aleatorio
Escribe un script que genere un arreglo de 10 números aleatorios, entre el 1 y el 100.

Ejercicio 10: Crear una calculadora
Escribe un script que simule una calculadora básica, donde el usuario pueda ingresar dos números y elegir la operación a realizar (+, -, *, /). Luego, muestra el resultado en pantalla.

Ejercicios con archivos

Ejercicio 1: Crear un archivo de texto

- Crea un archivo de texto llamado "miarchivo.txt".
- Escribe una frase de tu elección en el archivo.
- Guarda y cierra el archivo.

Ejercicio 2: Leer un archivo de texto línea por línea

- Abre el archivo "miarchivo.txt".
- Lee y muestra el contenido del archivo línea por línea.
- Cierra el archivo.

Ejercicio 3: Escribir en un archivo de texto

- Abre el archivo "miarchivo.txt".
- Escribe una nueva línea de texto en el archivo.
- Guarda y cierra el archivo.

Ejercicio 4: Mover un archivo a una carpeta diferente

- Mueve el archivo "miarchivo.txt" a una carpeta llamada "archivos/otracarpeta".
- Verifica que el archivo se haya movido correctamente.

Ejercicio 5: Crear una estructura de directorios

- Crea una carpeta llamada "archivos/nuevodirectorio".
- Dentro de "nuevodirectorio", crea un archivo de texto llamado "archivo1.txt" y escribe algo en él.
- Crea una subcarpeta dentro de "nuevodirectorio" llamada "subcarpeta".
- Dentro de "subcarpeta", crea otro archivo de texto llamado "archivo2.txt" y escribe algo en él.

Ejercicio 6: Eliminar un directorio y su contenido de forma recursiva

- Elimina de forma recursiva el directorio "archivos/nuevodirectorio" y todo su contenido.
- Verifica que el directorio y su contenido se hayan eliminado correctamente.

Estos ejercicios prácticos te permitirán aplicar los conceptos que hemos discutido, como la creación y manipulación de archivos, la lectura y escritura de archivos, el movimiento de archivos, la creación de directorios y la eliminación recursiva de directorios.